

SMC Scratchpad RAM — Functional Specification

SMC Scratchpad RAM — Functional Specification

Contents

1. Purpose & scope
2. Conformance & references
3. Feature summary
4. Configuration parameters
5. Block diagram & port list
6. Theory of operation
7. Memory map
8. Functional behaviour
 - 8.1 Reads
 - 8.2 Writes
 - 8.3 Preload contract
 - 8.4 Reset behaviour
9. Reset behaviour
10. Bus interface (TLM-2.0 / TileLink-style)
 - 10.1 Socket type
 - 10.2 Transaction contract
 - 10.3 Timing annotation
 - 10.4 `transport_dbg`
11. Error handling
12. ECC (SECEDED) model
13. Preload formats
 - 13.1 Hex format (`init_file_format = "hex"`)
 - 13.2 Binary format (`init_file_format = "bin"`)
 - 13.3 `auto` resolution
 - 13.4 Zero-padding rules
14. Compliance matrix
 - 14.1 Documented deviations
 - 14.2 Unmodelled features
15. Revision history
16. Glossary

SMC Scratchpad RAM — Functional Specification

Document: 01_SCRATCHPAD_RAM_Specification.md **Module:** smc::scratchpad_ram (SystemC/TLM-2.0 Loosely-Timed model, CCI-parameterised) **IP family:** SMC IP library (sister to smc::bootrom, smc::plic, smc::clint) **Spec base:** tt-oca-hw/hw/smc/smc_cpu/data/registers/rdl/spm_memory.rdl and the generated TileLink SRAM (WithScratchpadWithECC) **Status:** Frozen for SMC bring-up; tracks the chipyard scratchpad config and the generated 0CAH*CORECluster_TLRAM.sv RTL **Companion docs:** -
02_SCRATCHPAD_RAM_LowLevel_Design.md — internal SystemC implementation -
03_SCRATCHPAD_RAM_Test_Plan.md — verification strategy & test list

Contents

1. Purpose & scope
 2. Conformance & references
 3. Feature summary
 4. Configuration parameters
 5. Block diagram & port list
 6. Theory of operation
 7. Memory map
 8. Functional behaviour
 9. Reset behaviour
 10. Bus interface (TLM-2.0 / TileLink-style)
 11. Error handling
 12. ECC (SECDED) model
 13. Preload formats
 14. Compliance matrix
 15. Revision history
 16. Glossary
-

1. Purpose & scope

The **SMC Scratchpad RAM** is an on-chip, byte-addressable, read-write SRAM in the System Management Controller (SMC) CPU cluster. The SMC Rocket RV64GC cores use it as early-boot stack and data storage before external DRAM is trained, and as a fast local scratch region thereafter.

In the RTL it is generated by the chipyard testchipip.soc.WithScratchpadWithECC fragment as a TileLink SRAM with SECDED ECC, attached to the system bus (SBUS):

```
new testchipip.soc.WithScratchpadWithECC(base = 0xC0040000L,
    size = 0x00010000L,    // 64 KiB
    banks = 1,
    partitions = 4,
    ecc = ECCParams(bytes = 8,
        code = SECCDED),
    busWhere = SBUS)
```

This specification defines the **externally-observable behaviour** of the SystemC model: configuration, memory map, access semantics, byte-enable handling, error responses, the SECCDED ECC abstraction, reset, and bus contract. It is the read-write counterpart to the Boot ROM specification.

Internal implementation choices (data layout, byte-enable loop, the ECC error-flag map, CCI scaffolding) are documented in [02_SCRATCHPAD_RAM_LowLevel_Design.md](#).

2. Conformance & references

Source	Authority
tt-oca-hw/hw/smc/smc_cpu/data/registers/rdl/spm_memory.rdl	Ground-truth memory declaration (mem -type, 64-bit wide, sw=rw, hw=rw)
tt-oca-hw/hw/smc/smc_cpu/chipyard_config/OCAH1COREcluster.scala	Scratchpad geometry: base 0xC0040000, size 0x10000, banks 1, partitions 4, SECCDED
tt-oca-hw/hw/smc/smc_cpu/chipyard_generated_files/1core/OCAH1COREcluster_TLRAM.sv	Generated TileLink SRAM RTL (byte-mask writes, SECCDED, RMW, atomics)
tt-oca-hw/hw/smc/data/registers/rdl/smc_top.rdl	SMC top-level address map (spm_memory @ BASE_ADDR + 0x06_0000)
01_BOOTROM_Specification.md, 01_PLIC_Specification.md, 01_CLINT_Specification.md	Sister SMC IPs — same conventions

The SystemC model is **byte-identical** to the RDL memory layout: a contiguous NUM_ENTRIES -by-8-byte memory at offset 0, addressable at any naturally-aligned {1, 2, 4, 8} -byte width, with per-byte write masks.

A note on size. The spm_memory.rdl default NUM_ENTRIES describes a generic 1 MiB instance; the generated 1-core scratchpad is 64 KiB (size = 0x10000). The model's size_bytes is a CCI parameter so a single model serves either geometry; the default is 64 KiB to match the generated 1-core RTL.

3. Feature summary

Feature	Value / behaviour
Standard	OCA SMC scratchpad RAM; <code>mem</code> block in RDL, TileLink SRAM in RTL
Size	64 KiB (default; configurable via CCI <code>size_bytes</code>)
Native word width	64 bits (matches <code>regwidth</code> = <code>memwidth</code> = 64 in the RDL)
Supported access widths	1, 2, 4, 8 bytes (naturally aligned)
Software access	Read-write (<code>sw=rw</code> , <code>hw=rw</code>)
Byte-enables	Honoured — per-byte write masks (RTL uses an 8-bit mask + RMW)
Read-after-write	Returns the just-written value
Preload formats	Optional: hex (text, big-endian) or binary (raw little-endian)
Zero-init behaviour	When <code>init_file</code> is empty: all bytes start <code>0x00</code> (cold boot)
ECC	SECDED behavioural model (single-bit scrub, double-bit error)
Bus interface	TLM-2.0 LT target socket (AXI4 / TileLink-style)
DMI	Not granted (a DMI write would bypass SECDED re-encode; future work)
Reset polarity	Active-low, synchronous; contents retained (SRAM)
Configuration interface	SystemC CCI 1.0 (<code>cci::cci_param<T></code>)
Annotated TLM delay	2.0 ns default (pipelined ~2-cycle read); mutable via <code>access_delay_ns</code>

4. Configuration parameters

The model exposes **five real** `cci::cci_param<T>` **declarations**. Size, preload, and ECC-enable are immutable (changing them after elaboration would invalidate the in-memory image); the access delay is mutable so simulations can sweep bus speeds without rebuilding.

Parameter	Type	Default	Mutability	Notes
<code>size_bytes</code>	<code>uint64_t</code>	<code>0x10000</code>	immutable	Must be a non-zero multiple of 8 (the native 64-bit word width).
<code>init_file</code>	<code>std::string</code>	<code>""</code>	immutable	Optional preload image; empty = zero-fill (cold-boot state).
<code>init_file_format</code>	<code>std::string</code>	<code>"auto"</code>	immutable	<code>"hex"</code> , <code>"bin"</code> , or <code>"auto"</code> (pick <code>bin</code> for <code>.img/.bin</code> , else <code>hex</code>).
<code>access_delay_ns</code>	<code>double</code>	<code>2.0</code>	mutable	TLM <code>b_transport</code> annotated delay; pipelined read ≈ 2 cycles at 1 GHz.
<code>ecc_enabled</code>	<code>bool</code>	<code>true</code>	immutable	Enables the SECDED error-injection / single-bit-scrub behaviour.

Override before construction via the CCI broker:

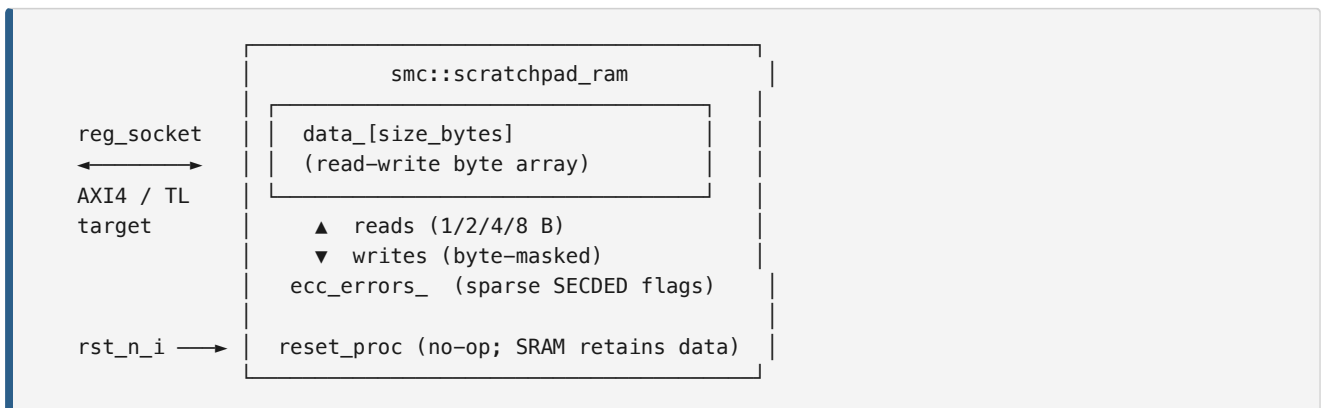
```
broker.set_preset_cci_value("smc.scratchpad_ram.size_bytes",
                           cci::cci_value(uint64_t(0x10000)));
broker.set_preset_cci_value("smc.scratchpad_ram.init_file",
                           cci::cci_value(std::string("scratch.hex")));
broker.set_preset_cci_value("smc.scratchpad_ram.access_delay_ns",
                           cci::cci_value(2.0));
broker.set_preset_cci_value("smc.scratchpad_ram.ecc_enabled",
                           cci::cci_value(true));
```

The constructor still accepts a `scratchpad_ram_cfg` struct; its fields supply **defaults** for the corresponding CCI params, so code that does not use the CCI broker continues to work unchanged.

Fixed structural constants:

Constant	Value	Description
WORD_BYTES	8	Native word width and ECC-protection granule (from RDL <code>memwidth = 64</code>)
MAX_ACCESS_BYTES	8	Largest single <code>b_transport</code> access width accepted

5. Block diagram & port list



Port	Direction	Type	Description
reg_socket	target	<code>tlm_utils::simple_target_socket<scratchpad_ram></code>	TLM-2.0 target socket; AXI4 / TileLink-style read-write access
rst_n_i	input	<code>sc_in<bool></code>	Active-low synchronous reset (does not clear SRAM contents)

The Scratchpad RAM has **no output ports**. It is a passive memory target.

6. Theory of operation

The scratchpad is a contiguous byte array (`size_bytes` long, default 64 KiB) that serves `b_transport` reads and writes verbatim. Unlike the Boot ROM, writes **commit** — it is a true RAM.

Three properties tie the model to the RTL:

1. **Byte-masked writes.** The TileLink SRAM carries an 8-bit byte mask (`auto_in_a_bits_mask[7:0]`). Sub-word and partial writes are committed per byte; the RTL realises this with a read-modify-write so the SECDDED code stays consistent across the whole 64-bit word. The model honours the TLM byte-enable array to the same effect.
2. **SECDDED ECC.** Each 64-bit word is protected by a single-error-correct, double-error-detect code (8 ECC bits over 64 data bits → 72-bit storage in RTL). Single-bit errors are corrected transparently on read; double-bit errors are flagged on the TileLink `d` channel (`d_corrupt`). The model reproduces this behaviourally (see §12).
3. **Pipelined access.** The RTL read path is registered (request → SRAM → response), giving a ~2-cycle latency. The LT model annotates this as a configurable `access_delay_ns` (default 2 ns at a 1 GHz fabric clock).

7. Memory map

All offsets are byte offsets relative to the scratchpad base address (`0xC0040000` in the 1-core SMC address map; the model works in local offsets, with the bus subtracting the base).

Range	Region	Access	Size
<code>0x00000 .. size_bytes - 1</code>	<code>mem_array[0..N - 1]</code>	R/W	8 B × N (default N = 0x2000 → 64 KiB)
<code>≥ size_bytes</code>	Out-of-window	—	TLM_ADDRESS_ERROR_RESPONSE

The default `size_bytes = 0x10000` matches the generated 1-core scratchpad (`WithScratchpadWithECC size = 0x10000`).

8. Functional behaviour

8.1 Reads

A read at byte offset `0` with width `w ∈ {1, 2, 4, 8}` returns the byte slice `data_[0..0+w - 1]`. Multi-byte reads honour native CPU endianness — the model never byte-swaps.

Property	Behaviour
Alignment	Naturally aligned to <code>W</code> bytes
Width range	$W \in \{1, 2, 4, 8\}$
Byte-enables	Honoured: only enabled bytes are returned
Response status	<code>TLM_OK_RESPONSE</code> on success
Annotated delay	$\text{+}=\text{ access_delay_ns} \times \text{ns}$
Side effects	A correctable ECC error on a covered word is scrubbed (see §12)

8.2 Writes

A write at byte offset `0` with width `W` and in-range `0 + W ≤ size_bytes`:

Property	Behaviour
Effect on <code>data_</code>	Enabled bytes committed; disabled bytes unchanged
Byte-enables	Honoured: per-byte write mask
Response status	<code>TLM_OK_RESPONSE</code>
Annotated delay	$\text{+}=\text{ access_delay_ns} \times \text{ns}$
ECC	Touched words are re-encoded (any injected error on them is cleared)
Subsequent read	Returns the just-written value

8.3 Preload contract

At construction time, if `init_file` is non-empty, the file is opened and decoded according to the resolved format (see §13). The decoded bytes are stored starting at offset 0; the remainder is left zero. If `init_file` is empty the entire RAM is zero-initialised — the usual cold-boot state.

Preload errors (file not found, unreadable, malformed) are reported with `SC_REPORT_FATAL` at elaboration time, so an integration mistake fails fast.

8.4 Reset behaviour

Reset is a **structural no-op for the array**: real SRAM retains its contents across a logical reset, and the RTL only resets the TileLink bus pipeline registers (`d_full` / `r_full`). The `reset_proc SC_METHOD` is registered for IP-suite symmetry — every other SMC IP exposes a `rst_n_i`.

9. Reset behaviour

Signal	Polarity	Edge	Effect on RAM state
<code>rst_n_i</code>	active-low	level-low	None (SRAM retains contents)

A wired-low `rst_n_i` does not stall the bus interface — reads and writes continue to be served throughout the reset window in the LT model.

10. Bus interface (TLM-2.0 / TileLink-style)

10.1 Socket type

```
tlm_utils::simple_target_socket<scratchpad_ram> reg_socket;
```

Registered callbacks:

```
reg_socket.register_b_transport (this, &scratchpad_ram::b_transport);
reg_socket.register_transport_dbg(this, &scratchpad_ram::transport_dbg);
```

DMI is **deliberately not registered**. A RAM is a strong DMI candidate, but a DMI write would bypass the SECDED re-encode path, so we keep it un-granted for parity with the other SMC IPs (see [02_SCRATCHPAD_RAM_LowLevel_Design.md §16](#) for the future-work item).

10.2 Transaction contract

Field	Required value	Notes
command	TLM_READ_COMMAND / TLM_WRITE_COMMAND	TLM_IGNORE_COMMAND → TLM_COMMAND_ERROR_RESPONSE
address	aligned to data_length, < size_bytes	Otherwise error (see §11)
data_length	1, 2, 4, or 8	Otherwise TLM_BURST_ERROR_RESPONSE
streaming_width	== data_length	Otherwise TLM_BURST_ERROR_RESPONSE
byte_enable_ptr	optional	When set, selects which bytes are read/written (per-byte mask)
dmi_allowed	(cleared on every transaction)	DMI never granted

10.3 Timing annotation

`b_transport` adds `sc_time(access_delay_ns_p.get_value(), SC_NS)` to the supplied `delay`. The model re-reads the CCI param on every transaction, so a broker-driven mutation takes effect on the next access.

`transport_dbg` adds nothing — debug accesses are zero-time by TLM-2.0 convention.

10.4 transport_dbg

Side-effect-free for timing, but **read-write** like `b_transport` (debug writes commit). Debug accesses **bypass ECC** (a debugger sees raw memory). On validation error `transport_dbg` returns 0; on success it returns the number of bytes transferred.

11. Error handling

Condition	Detection	Response
<code>data_length ∉ {1, 2, 4, 8}</code>	<code>b_transport</code> precheck	<code>TLM_BURST_ERROR_RESPONSE</code>
Address not naturally aligned to <code>data_length</code>	<code>b_transport</code> precheck	<code>TLM_BURST_ERROR_RESPONSE</code>
<code>streaming_width ≠ data_length</code>	<code>b_transport</code> precheck	<code>TLM_BURST_ERROR_RESPONSE</code>
<code>address ≥ size_bytes</code> or <code>address + len > size_bytes</code>	<code>b_transport</code> window check	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
Unsupported command (<code>TLM_IGNORE_COMMAND</code>)	<code>b_transport</code> switch	<code>TLM_COMMAND_ERROR_RESPONSE</code>
Read of a word with an uncorrectable ECC error	<code>b_transport</code> ECC check	<code>TLM_GENERIC_ERROR_RESPONSE</code>
Construction with <code>size_bytes = 0</code> or non-multiple of 8	Constructor	<code>SC_REPORT_FATAL</code> at elaboration
Construction with <code>access_delay_ns < 0</code>	Constructor	<code>SC_REPORT_FATAL</code> at elaboration
Preload file missing / unreadable / empty	<code>load_preload()</code>	<code>SC_REPORT_FATAL</code> at elaboration
Malformed hex line	<code>parse_hex_line()</code>	<code>SC_REPORT_FATAL</code> (file + line no.)
Hex preload overflow with non-zero data	<code>load_preload()</code>	<code>SC_REPORT_FATAL</code>
Hex preload overflow with all-zero tail	<code>load_preload()</code>	Tolerated (truncated silently)

`SC_REPORT_FATAL` is reserved for elaboration-time misconfiguration; runtime failures use TLM response codes so the surrounding fabric and test bench see deterministic bus-level errors.

12. ECC (SECDED) model

The RTL scratchpad protects each 64-bit word with a SECDED code. A bit-accurate ECC model is out of scope for an LT functional model; instead the model provides a **behavioural abstraction** controlled by the `ecc_enabled` CCI parameter and a debug error-injection API.

API / event	Behaviour
<code>dbg_inject_ecc_error(off, correctable=true)</code>	Marks the covering word as having a single-bit error.
Read covering a correctable word	Succeeds; data returned intact; the flag is scrubbed (matches SECDDED auto-correct on access).
<code>dbg_inject_ecc_error(off, correctable=false)</code>	Marks the covering word as having an uncorrectable (double-bit) error.
Read covering an uncorrectable word	TLM_GENERIC_ERROR_RESPONSE (LT analogue of TileLink <code>d_corrupt</code>).
Any write to a word	Re-encodes the word; clears its error flag.
<code>dbg_clear_ecc_errors()</code>	Drops all injected error flags.
<code>ecc_enabled = false</code>	Injection is a no-op; the model behaves as a plain RAM.

With no injected errors (the default) the model behaves as a plain RAM. The error flags are stored sparsely (a hash map keyed by word offset), so there is no per-word memory or time overhead in the common case.

13. Preload formats

A preload is **optional** for a scratchpad (it normally cold-boots to zero), but is supported for deterministic test fixtures. The format is identical to the Boot ROM's preload pipeline.

13.1 Hex format (`init_file_format = "hex"`)

Plain text, one 64-bit word per line, big-endian ASCII hex (most significant nibble first). Compatible with `xxd -r -p` round-tripping. Blank lines and `#`-prefixed comments are ignored; an optional `0x / 0X` prefix is accepted. Each line is stored little-endian at the next consecutive 8-byte slot.

13.2 Binary format (`init_file_format = "bin"`)

Raw binary, dumped to disk as a contiguous byte stream. Each 8 bytes on disk form one 64-bit word, stored at the matching offset without any swap.

13.3 `auto` resolution

The default `init_file_format = "auto"` picks `bin` if the filename ends in `.img` or `.bin` (case-insensitive), and `hex` otherwise. Any other value falls back to the same auto-detection rule.

13.4 Zero-padding rules

File size	Preload result
0 bytes	SC_REPORT_FATAL (empty preload is treated as an error)
< size_bytes	Loaded into data_[0..N-1] ; the remainder is zero
= size_bytes	Exact fit
> size_bytes (hex)	Tolerated iff every overflow word is zero; non-zero overflow word → SC_REPORT_FATAL
> size_bytes (bin)	Truncated silently

14. Compliance matrix

Claim	Source	Statement	Model behaviour
—	RDL sw=rw, hw=rw	RAM is software read-write	Reads return stored data; writes commit
—	RDL memwidth = 64	64-bit native word width	Default size_bytes is NUM_ENTRIES × 8 ; 8-byte access is the maximum
—	RTL auto_in_a_bits_mask[7:0]	Per-byte write mask (sub-word writes via RMW)	Byte-enables honoured on read and write
—	RTL SECDDED ECC (d_corrupt)	Single-bit corrected, double-bit flagged	Behavioural SECDDED model (§12)
—	RTL pipelined read	~2-cycle read latency	access_delay_ns default 2 ns
—	RTL reset	SRAM array not cleared by reset	reset_proc is a no-op; contents retained
—	SMC IP convention	All SMC IPs expose rst_n_i	rst_n_i registered as SC_METHOD

14.1 Documented deviations

Item	Reason
Atomic memory operations (AMOs)	The TileLink SRAM supports arithmetic/logical atomics via its RMW ALU; the LT model treats writes as plain stores. Future work.
Bit-accurate SECDDED	Modelled behaviourally (injection API) rather than a true Hamming code.
DMI fast-path	Un-granted: a DMI write would bypass the SECDDED re-encode. Future work.

14.2 Unmodelled features

Item	Reason
Approximately-Timed (AT) protocol	LT model is sufficient for SW bring-up.
Per-bank / per-partition timing	The RTL splits the array into 4 partitions; the model is a single flat array (functionally identical).
ECC error counters / interrupts	<code>notifyErrors = false</code> in the RTL config; errors are reported on the bus only.

15. Revision history

Version	Date	Author	Notes
1.0	2026-06-03	SMC modelling	Initial release — read-write SRAM with byte-enables and a behavioural SECEDED model; tracks <code>spm_memory.rdl</code> and the generated <code>WithScratchpadWithECC</code> TileLink SRAM.

16. Glossary

- **Scratchpad RAM** — A small, on-chip, byte-addressable read-write memory used by a processor for fast local storage.
- **SECEDED** — Single-Error-Correct, Double-Error-Detect ECC code; corrects one bit error and detects (but cannot correct) two.
- **Byte-enable** — A per-byte write mask; lets a transaction update a subset of the bytes in a word.
- **RMW (Read-Modify-Write)** — How the RTL applies a sub-word write under an ECC code: read the word, merge the new bytes, re-encode ECC, write back.
- **TileLink** — The on-chip bus protocol used by the Rocket/chipyard CPU cluster; the scratchpad is a TileLink SRAM target.
- **DMI** — TLM-2.0 Direct Memory Interface; a back-door pointer for fast access (not granted by this model).
- **LT (Loosely-Timed)** — TLM-2.0 modelling style: `b_transport`, optional DMI, coarse `sc_time` annotations.
- **RDL** — SystemRDL register-description language; the authoritative scratchpad declaration lives in `spm_memory.rdl`.
- **CCI** — SystemC Configuration, Control and Inspection (Accellera cci-1.0.0). Provides the `cci_param<T>` interface used here.
- **SMC** — System Management Controller; the RV64GC management complex whose IP library this model belongs to.
- **SC_REPORT_FATAL** — SystemC's mechanism for reporting an unrecoverable configuration error during elaboration.

End of document.